

High-Level Design Document for <project name> project

Author	Ido Talker and Itay Elam
Version	V0.1
Created	14/06/2026
Last Updated	18/06/2026

1 Introduction

1.1 Overview

This document details our planned implementation of parallel transaction execution in Geth. The main idea is to take the regular block execution process, which currently runs transactions one after another, and add a parallel execution path for transactions that do not depend on each other. We will assume that the needed dependency information is declared, use it to split the transactions into execution waves, run each wave in parallel, and then commit the state changes back into Geth in a controlled way. The document explains this implementation, the main modules, the execution flow and the assumptions we rely on.

1.2 Assumptions and Design Constraints List

Our main assumptions can be split into two categories.

- 1) We assume that transactions declare the full read and write list and that we know which accounts are touched in each transaction. This is essential as without this we cannot guarantee out of order correctness.
- 2) There is a shared resource called the "coinbase". This is a shared resource between all transactions which can be added to dynamically and can be conditioned on using control flow. We generally cannot predict if and how much a transaction might add to it, and we also don't currently see a good way to validate if a transaction might condition on it. In our recent analysis, this seems to be a leftover from the past and not something that is widely used and so we believe this assumption will hold in the real world as well.

1.3 Distributed & Parallel Aspects

The distributed and parallel aspects are important in our project because the goal is to make Geth execute independent Ethereum transactions in parallel instead of always executing the entire block sequentially. This is not inherently parallel in the simple sense, because Ethereum was designed as a sequential state machine and Geth normally processes transactions one after another. However, some transactions inside a block are only partially dependent on each other, so if their dependencies can be identified, the block execution can be made efficiently parallelizable.

The main difficulties compared to a sequential implementation are detecting which transactions depend on each other and handling duplicated states efficiently. A naive implementation that copies the entire state for every transaction would be unnecessarily expensive, so the implementation needs to copy and merge only what is actually needed. In this project we deal with dependency detection by assuming the required read/write information is declared in advance and we deal with efficient state handling through careful analysis and implementation of the copied state and merge logic. The main benefit of doing this is speedup: independent transactions can run concurrently, reducing total block execution time while still preserving the same result as sequential execution.

1.4 Dependencies

None

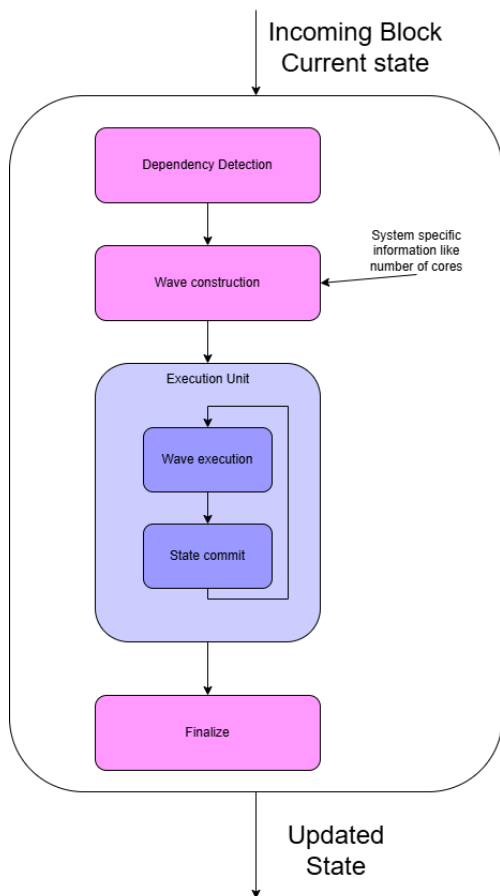
1.5 Issues List

- We recently discovered Geth correctness tests and most pass but some seem to fail. Our primary task is to identify the failure cases and correct them.
- Improve our launch state – we currently do a large state copy for each transaction run, this is probably copying unneeded information, we might be able to slim it down or at least pre-allocate a pool of states, improving the launch speed of each wave.
- Improve our transaction to wave scheduling, we currently do not consider the hardware limits when assigning transactions to waves, which might mean we attempt to launch too many transactions at once, overwhelming the CPU and reducing overall throughput.

1.6 To-do List + Expected time-tables

TODO	Expected time
Run, validate and fix correctness with Geth Tests	19/07/2026
Improve launch state management	15/08/2026
Improve transaction scheduling	30/08/2026

2 Parallel Execution Model and State Management



2.1 Dependency Detection

This module identifies dependencies between transactions based on their declared read/write sets and accounts they interact with. Its goal is to determine which transactions may conflict and require sequential execution (with respect to each other) and which transactions can safely be executed in parallel.

2.2 Wave Construction

This module groups transactions into execution waves. Transactions inside the same wave are considered independent and can run concurrently, while dependencies between waves preserve the required execution order. System information, such as the number of available cores, can also affect the size and structure of the waves.

2.3 Execution Unit

This module is responsible for executing each wave and committing its result. It separates the two logical steps, parallel execution phase and state update phase. We will additionally examine if

transactions can run concurrently while state changes are still applied in a controlled and deterministic way.

2.4 Wave Execution

This module executes all transactions in the current wave in parallel. Each transaction runs with isolated state handling so that concurrent execution does not directly mutate the shared global state.

2.5 State Commit

This module merges the results of the executed wave back into the main state. The merge must be done carefully and deterministically – we need to make sure we do not overwrite data when merging multiple state as to preserve the behavior of sequential execution.

2.6 Finalize

This module completes the block execution after all waves have been processed. It finalizes the resulting state, receipts, logs, and other execution outputs so they match the expected Geth block execution format.

3 Design

3.1 Modules / Main Classes

By the nature of our project, we aim to stick as much as possible to the current design of Geth, this approach will be beneficial in case we move forward with our work and introduce it to the community.

The main classes we forecasting to change are –

- **StateProccecor**: Responsible for the main execution loop.
- **StateDB**: Responsible for state management, some parts of it will be have to modified in order to become thread safe (as per Roi's analysis).
- **BlockValidator**: Due to parallel execution it is possible that the current validator, which runs after each transaction may be changed in order to consider the new order, the current implementation assume strict ordering and sequential execution.

3.2 Flows

1. Initialization(Blockchain):

The flow begins when a batch of blocks is submitted via `Blockchain.InsertChain()`. This method performs basic contiguous linking checks and delegates to `insertChain()`, which iterates through the blocks. For each block, `processBlock()` is invoked.

2. State Preparation :

Inside `processBlock()`, the system fetches the parent block's state root and initializes a fresh `StateDB` instance to act as a scratchpad for the new block's state mutations.

3. Execution (StateProcessor)

The StateProcessor takes the block and the fresh StateDB, handles block-level forks and pre-execution system calls (like storing the parent block hash or beacon root in designated system contracts). It loops over all transactions in the block (potentially grouping them into parallel execution waves if address overlap permits).

For each transaction, it creates a vm.EVM instance, invokes ApplyTransactionWithEVM, and records the resulting Receipt (including logs, gas used, and status).

4. State Validation (BlockValidator):

Once all transactions are executed, the BlockChain takes the aggregated results (ProcessResult) and hands them to the BlockValidator.

The validator confirms that the GasUsed locally matches the GasUsed in the block header.

It merges all transaction receipts and verifies the ReceiptHash and Bloom filter.

Crucially, it asks the StateDB to calculate its IntermediateRoot (the new state root after all mutations) and ensures it perfectly matches the Root claimed in the block header. If any transaction execution differed from the global block validator, this root check will fail.

5. Commitment:

If validation succeeds, processBlock() writes the block, receipts, and state preimages to the underlying disk database and updates the canonical chain head.

3.3 Setup

Follow go-ethereum's [dev guide](#).

4 References - to external papers/packages

[go-ethereum](#) - The project is developed as a modification of Geth, the official Go implementation of the Ethereum protocol. We use Geth's existing block execution logic, EVM implementation, transaction processing, and StateDB infrastructure. Our contribution is adding a parallel execution layer around the existing transaction execution flow, including dependency-based wave construction, isolated state execution, and controlled state commit.